# 🎓 Guide de démonstration — Transport Scolaire Intelligent

### Ce qu'il faut dire au professeur (Pr. Ali Bekri)

> **Ordre imposé : 1) le pipeline Data Science d'abord → 2) la démonstration / les tests ensuite.** Ton domaine est la data science : c'est le cœur de la soutenance, donc on **raconte la démarche scientifique avant** de montrer que « ça marche ».

---

## ⏱️ Plan de la démo (≈ 12 min)

| Bloc | Durée | Objectif |
|---|---|---|
| 0. Contexte | 1 min | Poser le problème métier |
| 1. **Pipeline Data Science (CRISP-DM)** | 6 min | Montrer la rigueur scientifique |
| 2. **Démonstration live (tests)** | 4 min | Prouver que le système fonctionne |
| 3. Questions | 1 min | Anticiper les questions du jury |

---

## 0️⃣ Contexte — la phrase d'accroche (1 min)

> « Le problème : regrouper **300 élèves de Meknès** en **groupes affectables à un bus** — chaque groupe doit (1) tenir dans la **capacité du bus**, (2) être **géographiquement compact** pour minimiser le temps de ramassage, et (3) rester **robuste** quand les données évoluent. Ce n'est pas un simple `KMeans()` : c'est un problème de **clustering géospatial sous contrainte de capacité**. »

**À montrer :** la carte de la console avec les 300 élèves (onglet Clustering).

---

## 1️⃣ Pipeline Data Science — la démarche CRISP-DM (6 min)

> **Support : les 3 notebooks** dans `ms6-clustering/notebooks/`. Tu les fais défiler en racontant. Ne lis pas le code — raconte **les décisions** et **pourquoi**.

### Notebook 1 — `clustering_study.ipynb` (la démarche complète)

**a) Business understanding.**

> « L'objectif métier n'est pas "le meilleur score mathématique" mais "des groupes faisables pour un bus". J'ai traduit ça en contraintes : taille ≤ capacité, compacité minimale, équilibre des groupes. »

**b) Data understanding (EDA).**

> « Données : 300 élèves, 6 zones / ~20 secteurs de Meknès, avec une **destination école** (Al Massira), un **oued infranchissable** qui sépare des quartiers, et **5 % d'élèves dispersés** (cas réels d'isolés). »

- Montrer : carte de densité, histogramme distance-à-l'école, k-distance plot.

**c) Data preparation — LE point clé à défendre.**

> « Erreur classique : faire `StandardScaler(lat, lon)` puis distance euclidienne. À la latitude de Meknès (**33.9°N**), 1° de longitude ≠ 1° de latitude en mètres → **la géométrie est faussée**. J'ai donc **projeté en UTM Zone 30N (EPSG:32630)** avec `pyproj` → coordonnées **en mètres** → la distance euclidienne devient la **vraie distance terrain**, et le Silhouette devient **interprétable en mètres**. »

➡️ **C'est la phrase qui montre que tu es data scientist et pas juste utilisateur de sklearn.**

**d) Modeling.**

> « J'ai comparé plusieurs familles d'algorithmes, pas un seul. »

## Notebook 2 — `02_model_comparison.ipynb` (benchmark 11 algorithmes)

> « Étude comparative de **11 algorithmes** : KMeans, KMeans capacitaire, DBSCAN, HDBSCAN, GMM, Spectral, BIRCH, OPTICS, Mean-Shift, Agglomératif (Ward)… évalués sur **plusieurs critères**, pas un seul score. »

**e) Evaluation — multi-critères (jamais le Silhouette seul) :**

| Critère | Type | Pourquoi |
|---|---|---|
| Silhouette ↑, Davies-Bouldin ↓, Calinski-Harabasz ↑ | indices internes | qualité géométrique |
| Faisabilité capacité (% groupes ≤ 50) | **KPI métier** | un groupe de 53 = inutilisable |
| Compacité intra-cluster (m) | KPI métier | proxy temps de ramassage |
| Équilibre des tailles (CV) | KPI métier | éviter 1 bus plein / 1 bus vide |
| ARI / AMI vs quartiers | stabilité | cohérence avec la réalité terrain |

## Notebook 3 — `03_production_strategy.ipynb` (la stratégie retenue)

**f) Deployment — la décision finale, gelée dans `production_config.json` :**

> « En production je vais plus loin que le notebook : j'utilise la **vraie distance routière OSRM** (réseau de Meknès), parce que la distance à vol d'oiseau **sous-estime** le trajet réel d'un facteur **1,5 à 1,7×**. »

**Configuration de production réelle (à citer) :**

```
{
  "strategie": "k-médoïdes capacitaire sur distance routière OSRM",
  "capacite_bus": 50,
  "k_production": 7,
  "silhouette_routier": 0.433,
  "compacite_med_m": 725,   // médoïde ≤ 725 m en médiane
  "compacite_max_m": 2409,
  "taille_max": 50,          // contrainte capacité RESPECTÉE
  "faisable": true
}
```

> « Le nombre de bus n'est **pas fixé arbitrairement** : je le dérive d'une **courbe de Pareto** (qualité vs nombre de bus) → c'est une **aide à la décision** pour l'admin. »

**Honnêteté scientifique (à assumer devant le jury) :**

> « Le Silhouette de production est **0,43**, sous mon seuil interne de 0,5. C'est **normal et voulu** : la contrainte de capacité dégrade légèrement la pure compacité géométrique — c'est l'**arbitrage métier ↔ mathématique**. Sans contrainte, KMeans monte plus haut mais produit des groupes infaisables. »

➡️ Assumer ce chiffre = preuve de maturité scientifique. **Ne jamais cacher un résultat moyen.**

### Récapitulatif des décisions d'architecture DS (slide / phrase de synthèse)

| Décision | Raison data science |
|---|---|
| Projection UTM 30N (pas StandardScaler) | distance vraie en mètres à 33.9°N |
| Distance routière OSRM | la distance à vol d'oiseau sous-estime de 1,5–1,7× |
| k-médoïdes capacitaire | garantit groupe ≤ capacité bus |
| Affectation globale (LP / min-cost-flow) | faisabilité garantie par construction |
| DBSCAN → bruit = groupe 999 | isole les dispersés (ramassage individuel) |
| Sélection multi-critères + Pareto | pas de sur-optimisation d'un score unique |

---

## 2️⃣ Démonstration live — les tests (4 min)

> Support : `html_tests/test-console.html` (console unifiée, tous les microservices).

### Prérequis (à lancer AVANT la démo)

```
# 1. Infra
docker-compose up -d ; docker start osrm_meknes
# 2. Microservices (ordre)
# MS7(8087) → MS1(8081) → MS2(8082) → MS3(8083) → MS10(8090) → MS4(8084) → MS8(8080)
# 3. MS6 clustering
cd ms6-clustering ; uvicorn main:app --port 8086 --reload
```

> 💾 **Données en base : v2** (300 élèves Meknès, cohérentes avec les notebooks — école, oued, dispersés).

### Scénario de démonstration (dans l'ordre)

**Étape 1 — Santé du système.**

- Onglet 🩺 Santé → **Tester tout** → 7 pastilles ✅ vertes.

> « Architecture microservices : 6 services Java Spring Boot + 1 service Python FastAPI, derrière une API Gateway. »

**Étape 2 — Authentification (sécurité).**

- Login ADMIN ( `admin@transport.local` ).

> « JWT HMAC-SHA256, double validation (Gateway + service), RBAC ADMIN/CHAUFFEUR/PARENT. »

**Étape 3 — Le clustering en action (LE moment fort).**

- Onglet 🧩 Clustering → **POST /production** avec `appliquer=true` .
- La carte se rafraîchit → **les nuages de clusters apparaissent**.

> « Voici le pipeline du notebook **en production réelle** : chaque **nuage** (enveloppe convexe) = un groupe-bus, le 🏫 = l'école, les points **gris** = les élèves dispersés isolés (groupe 999, ramassage individuel). Chaque groupe respecte la capacité de 50. »

- Montrer aussi **POST /pareto** → « voici l'aide à la décision sur le nombre de bus ».

**Étape 4 — Temps réel (si le temps le permet).**

- Onglet 📍 GPS → **Charger positions bus** + WebSocket → bus qui bougent sur la carte.
- Onglet ⚠️ Incidents → signaler un incident → RESOLU → déclenche une 🔔 notification (MS4).

> « Flux événementiel asynchrone via RabbitMQ : un incident propage automatiquement une notification au parent. »

---

## 3️⃣ Questions probables du jury — réponses prêtes

**« Pourquoi pas simplement KMeans ? »**

> KMeans seul ignore la capacité des bus et fausse les distances en lat/lon brut. J'ai ajouté la projection métrique, la contrainte de capacité et la distance routière.

**« Pourquoi un Silhouette de 0,43 seulement ? »**

> C'est l'arbitrage capacité ↔ compacité. Sans contrainte je monte plus haut, mais avec des groupes infaisables pour un bus. J'optimise le **métier**, pas un score isolé.

**« Comment ça évolue si un élève déménage / arrive ? »**

> C'est le sprint S8 : 5 déclencheurs de recalcul (ajout élève, déménagement, nouveau bus, Silhouette dégradé, planification). RabbitMQ déclenche déjà un recalcul auto (avec `asyncio.Lock`).

**« La distance routière, comment ? »**

> Matrice 300×300 calculée via l'API `/table` d'OSRM (réseau OSM Maroc), en requêtes asynchrones par blocs, mise en cache (`osrm_dist_m.npy`).

**« Pourquoi un service Python séparé pour le clustering ? »**

> L'écosystème data science (scikit-learn, numpy, pyproj) est en Python. FastAPI s'intègre proprement au reste via REST + RabbitMQ, sans polluer les services Java.

---

## ✅ Les 3 phrases à ne PAS oublier

1. **« J'ai projeté en UTM 30N car en lat/lon brut la distance est géographiquement fausse à 33.9°N. »**
2. **« J'optimise une contrainte métier (capacité bus), pas seulement un score mathématique — d'où le k-médoïdes capacitaire sur distance routière OSRM. »**
3. **« Je choisis le nombre de bus par une courbe de Pareto : c'est une aide à la décision, pas un chiffre arbitraire. »**

---

### 📂 Artefacts à avoir ouverts pendant la démo

- `ms6-clustering/notebooks/clustering_study.ipynb` (démarche)
- `ms6-clustering/notebooks/02_model_comparison.ipynb` (benchmark)
- `ms6-clustering/notebooks/03_production_strategy.ipynb` (stratégie + Pareto)
- `ms6-clustering/notebooks/production_config.json` (config gelée)
- `html_tests/test-console.html` (démo live)